IRISH AND FINNISH HOSPITAL RECRUITMENT DATABASE

System Overview

This project implements a relational database designed to manage hospital recruitment across Ireland and Finland.

The system tracks:
• Hospitals
• Job positions
• Candidates
• Interviews
• Skills

Hospital
This table stores information about hospitals.

```sql
CREATE TABLE Hospital (
    hospital_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    address VARCHAR(200) NOT NULL,
    phone VARCHAR(15)
);
```

Key details:

- Used auto-incrementing integer ID as the PK.
- Stores the VARCHAR hospital name (assumed to be NOT NULL),
- Stores a VARCHAR address (Also NOT NULL, an existing hospital must be located somewhere)
- Stores a VARCHAR phone number.
- Each Hospital has a **one-to-many** relationship with JobPosition (one hospital can advertise many positions).
- Hospital names are not required to be unique, as multiple hospitals may share the same name.

hospital_id	name	address	phone
1	Connolly Hospital	Waterville, Blanchardstown, Dublin 15	016465000
2	Beaumont Hospital	Dublin 9	018387111
3	University Hospital Limerick	Dooradoyle, Limerick	061301000
4	Cork University Hospital	Wilton, Cork	0214924200
5	St. James's Hospital	James's St, Dublin 8	014162000
6	Espoon Sairaala	Sairaalantie 1, Espoo	0901234567
7	Mäntsälän Terveyskeskus	Keskustie 5, Mäntsälä	019123456
8	Helsingin Meilahden Sairaala	Meilahdentie 2, Helsinki	0912345678
9	Tampereen Keskussairaala	Teiskontie 35, Tampere	031234567
10	Turun Yliopistollinen Sairaala	Kiinamyllynkatu 4, Turku	021234567

Candidate

This table stores information about job applicants.

```sql
CREATE TABLE Candidate (
    candidate_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    surname VARCHAR(50) NOT NULL,
    address VARCHAR(200),
    phone VARCHAR(15) UNIQUE,
    email VARCHAR(100) UNIQUE
);
```

Key details:

- Candidates have a **many-to-many** relationship with Skills, through the Candidate_Skill junction table.
- Forename and surname are assumed to be NOT NULL.
- Addresses are not required to be unique, as multiple candidates may share an address.
- Phone or email may be NULL to accommodate incomplete applications.

Skill

This table stores information about skills required by job positions or possessed by candidates.

```sql
CREATE TABLE Skill (
    skill_id INT AUTO_INCREMENT PRIMARY KEY,
    skill_name VARCHAR(50) UNIQUE NOT NULL
);
```

Key details:

- Skills participate in two **many-to-many** relationships.
  - with Candidates (via Candidate_Skill)
  - with JobPositions (via Position_Skill).
- Auto-incrementing INT skill_id as PK
- Name of skill assumed NOT NULL and UNIQUE to ensure no duplicate and that every skill is named.

skill_id	skill_name
5	Administration
2	Cardiology
4	Emergency Care
8	IT
10	Management
1	Nursing
3	Pharmacy
9	Physiotherapy
6	Radiology
7	Surgery

JobPosition

This table stores information about vacancies.

```sql
CREATE TABLE JobPosition (
    position_id INT AUTO_INCREMENT PRIMARY KEY,
    position_type VARCHAR(50) NOT NULL,
    hospital_id INT NOT NULL,
    FOREIGN KEY (hospital_id) REFERENCES Hospital(hospital_id) ON DELETE CASCADE
);
```

Key Details:

- **Many-to-One** relationship with Hospital.
- position_type NOT NULL, as each job must have a title, and not made UNIQUE as multiple hospitals could advertise the role type.
- FOREIGN KEY hospital_id (Linking Positions to Hospitals) is NOT NULL, since a vacancy cannot exist without being linked to an existing hospital.
- ON DELETE CASCADE, so that if a hospital is deleted, all its job positions are removed.

position_id	position_type	hospital_id
1	IT Support Lead	4
2	Nurse	1
3	Pharmacist	2
4	Radiologist	6
5	Emergency Doctor	10
6	Hospital Manager	8
7	Lab Technician	7
8	Brain Surgeon	3
9	Physiotherapist	5
10	Receptionist	9

Interview

This table stores information about interviews conducted between candidates and job positions.

```
CREATE TABLE Interview (
    interview_id INT AUTO_INCREMENT PRIMARY KEY,
    position_id INT NOT NULL,
    candidate_id INT NOT NULL,
    interview_date DATE NOT NULL,
    interview_time TIME NOT NULL,
    offer_made BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (position_id) REFERENCES JobPosition(position_id) ON DELETE CASCADE,
    FOREIGN KEY (candidate_id) REFERENCES Candidate(candidate_id) ON DELETE CASCADE
);
```

Key details:

- FOREIGN KEYS, Candidate_id and Position_id, NOT NULL and link Interviews with Candidates and Positions.
- Candidate/Position to Interview are **one-to-many** (candidates and positions may be involved in multiple interviews).

- This results in **Candidate to JobPosition** becoming a **many-to-many relationship** via the interview table**.**
- ON DELETE CASCADE for Foreign Keys to ensure interviews are removed if the related candidates /job positions are deleted.
- interview_date / interview_time NOT NULL.

interview_id	position_id	candidate_id	interview_date	interview_time	offer_made
1	1	4	2024-01-15	09:00:00	1
2	2	1	2024-01-16	10:30:00	1
3	3	2	2024-01-17	14:15:00	0
4	4	5	2024-01-18	11:00:00	1
5	5	1	2024-01-19	15:45:00	0
6	6	11	2024-01-20	13:30:00	1
7	7	10	2024-01-21	16:00:00	0
8	8	3	2024-01-22	09:30:00	1
9	9	6	2024-01-23	14:00:00	0
10	10	4	2024-01-24	10:00:00	1
11	3	7	2024-01-25	11:30:00	1
12	1	8	2024-01-26	14:00:00	0
13	8	13	2024-01-27	10:15:00	1

Junction Tables

Two junction tables implement many-to-many relationships in the schema.

```sql
CREATE TABLE Candidate_Skill (
    candidate_id  INT ,
    skill_id INT ,
    PRIMARY KEY (candidate_id, skill_id),
    FOREIGN KEY (candidate_id) REFERENCES Candidate(candidate_id) ON DELETE CASCADE,
    FOREIGN KEY (skill_id) REFERENCES Skill(skill_id) ON DELETE CASCADE
);
```

```sql
CREATE TABLE Position_Skill (
    position_id INT,
    skill_id INT,
    PRIMARY KEY (position_id, skill_id),
    FOREIGN KEY (position_id) REFERENCES JobPosition(position_id) ON DELETE CASCADE,
    FOREIGN KEY (skill_id) REFERENCES Skill(skill_id) ON DELETE CASCADE
);
```

Candidate_Skill Key Details:

- Implements a many-to-many relationship between Candidate and Skill
- Composite primary key (candidate_id, skill_id)
- Both attributes are foreign keys
- ON DELETE CASCADE for referential integrity

Position_Skill Key details**:**

- Implements many-to-many relationship between JobPosition and Skill
- Composite primary key (position_id, skill_id)
- Both attributes are foreign keys
- ON DELETE CASCADE

These two tables link Skills with Candidates/Positions by ID. This will be very useful for our stored procedures (finding positions/candidates with certain skills).

All tables are populated using stored procedures located in `04_seed_data.sql`.

```
insert_hospital(IN p_name VARCHAR(100), IN p_address VARCHAR(200), IN p_phone VARCHAR(15))

insert_candidate(IN p_first_name VARCHAR(50), IN p_surname VARCHAR(50), IN p_address
VARCHAR(200), IN p_phone VARCHAR(15), IN p_email VARCHAR(100))

insert_skill(IN p_skill_name VARCHAR(50))

insert_jobposition(IN p_position_type VARCHAR(50), IN p_hospital_id INT)

insert_candidate_skill(IN p_candidate_id INT, IN p_skill_id INT)

insert_position_skill(IN p_position_id INT, IN p_skill_id INT)

insert_interview(IN p_position_id INT, IN p_candidate_id INT, IN p_interview_date DATE, IN
p_interview_time TIME, IN p_offer_made BOOLEAN)
```

Data can be queried using stored procedures defined in `05_queries.sql`.

Parametric procedures were used for searches requiring specific input values

Referential Integrity

All foreign key relationships use ON DELETE CASCADE to maintain referential integrity.

This ensures that when a parent record is deleted, related records are automatically removed. For example, if a hospital is deleted, any job positions associated with that hospital are also removed.

Entity-Relationship Diagram

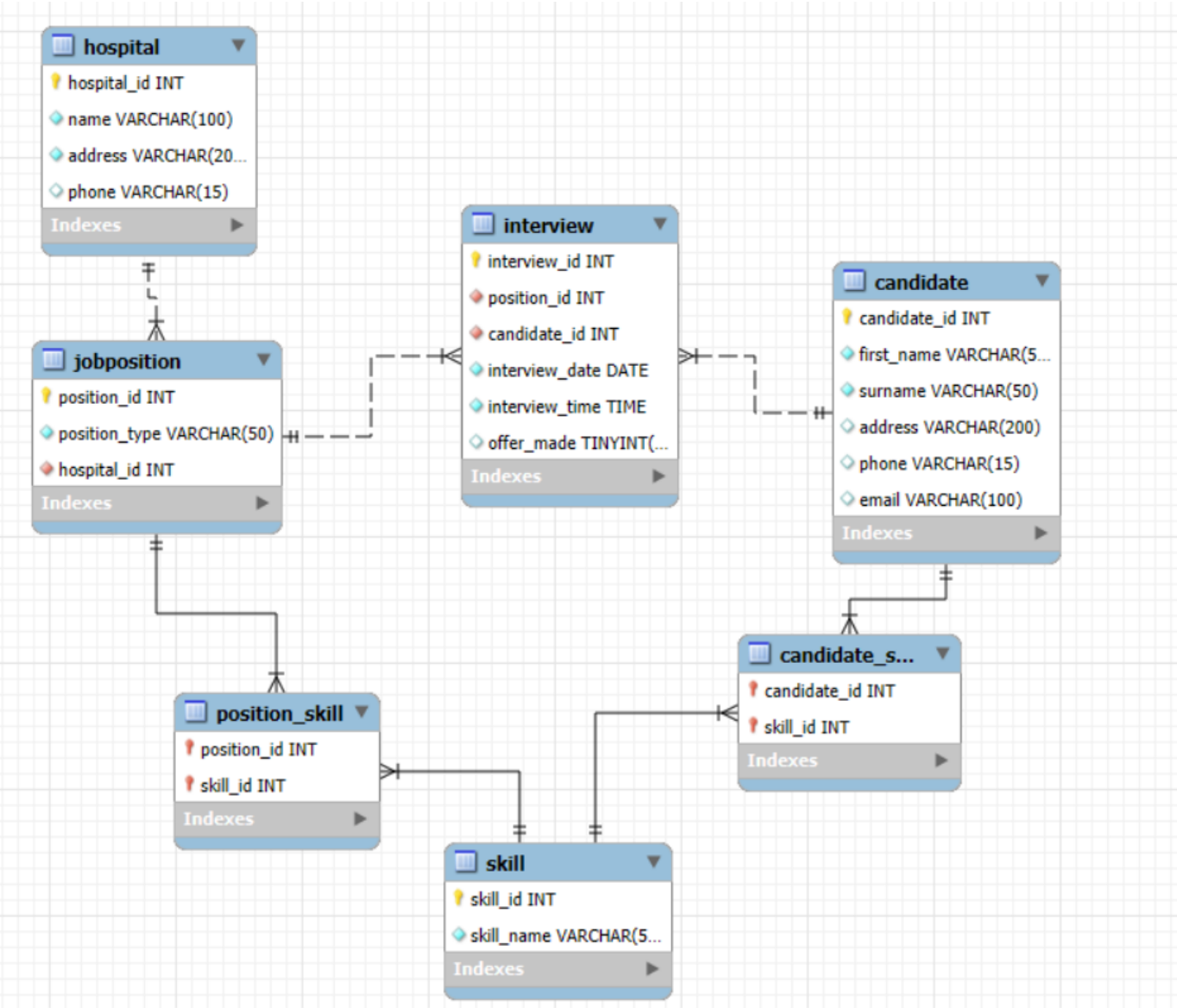

Figure: Entity-Relationship diagram generated using MySQL Workbench.

Development Environment

This database was developed and exported using MySQL Workbench 8.0 on Windows 11.